SPECIFICATION - VERSION 1.0 - 26 OCTOBER 2026

Android Companion App Specification

A small, sideloaded Android app for the Pixel 10 Pro XL that gives the AI agent Almusca three capabilities the Muse app does not offer: taking a photo on request, recording a short audio clip on request, and fetching existing photos from the gallery - reachable only over Tailscale, authenticated, and fully auditable on the device.

Working name	Almusca Device Bridge
Target device	Google Pixel 10 Pro XL (Rahul's phone), Android 14 or later
Owner	Rahul Sonnad - sole user, sole device
Distribution	Sideloaded APK only. No Play Store listing, no other users.
Status	Specification only. No app code is written under this document.

What this document decides

The scope of version 1, the system architecture, the exact constraints the Android platform imposes on camera and microphone capture, the local API the agent will call, the security and consent model, and the build, signing, install and pairing plan. Section 8 lists the decisions that remain open for Rahul; two of them must be answered before the build starts.

Reading note on honesty about the platform

Modern Android is designed to prevent exactly the kind of quiet background capture this app could be mistaken for. Section 3 states those limits plainly: capture always runs through a visible foreground service, the system shows its green camera and microphone indicators while a sensor is live, and several permissions need a one-time tap from Rahul on the phone itself. The app is designed to work with those signals, never around them. Anything this specification promises is achievable inside those rules; anything outside them is listed as a non-goal.

1. Purpose and scope

1.1 Why this app exists

The Muse Android app already gives Almusca a useful set of device commands: reading and sending SMS, location, calendar, contacts, call log, notifications and reminders. It offers no access to the camera, the microphone or the photo gallery. Three recurring needs fall into that gap: showing the agent something in the physical world (a product label, a room, a document on a table), capturing a short spoken note or ambient clip on request, and retrieving a photo Rahul already took instead of asking him to upload it manually.

The companion app exists to close exactly that gap. It is a private tool for one person and one phone, not a product.

1.2 Version 1 scope - in

Capability	What v1 does
Camera snapshot	On an authenticated request, open the selected camera (rear by default, front on request), capture a single still photo at up to full sensor resolution, and return it to the agent. Optional flash control. No video.
Microphone clip	On request, record one audio clip of a caller-specified length, capped at 120 seconds, in AAC/M4A, and return the file. Recording always starts and stops inside a single request.
Gallery fetch	List recent photos with metadata (timestamp, dimensions, size) and fetch individual photos by ID, newest first. Read-only; the app never deletes or modifies gallery items.
Status and audit	Report health, enabled capabilities and battery state; expose the on-device audit log of every access.

1.3 Explicit non-goals for v1

- No video recording and no live camera or audio streaming.
- No continuous or scheduled capture, and no capture triggered by anything other than an authenticated agent request.
- No capture while the request path cannot show the required foreground-service notification (see Section 3).
- No access to messages, contacts, files outside the photo gallery, or any other app's data - the Muse app already covers what is needed there.
- No multi-device support, no second user, no cloud relay, no analytics.

1.4 Later phases (not in v1)

Phase	Candidate capabilities	Notes
Phase 2	Device sensors snapshot (accelerometer, gyroscope, light, pressure); device state bundle (battery, network, storage); flashlight toggle	Low sensitivity; sensors still get per-capability toggles and audit entries.
Phase 3	Screen capture (single screenshot on request)	Requires Android MediaProjection, which needs an on-screen consent prompt from Rahul for each session. The API must handle "consent not granted" as a normal outcome, never a failure to work around.
Phase 3	Short video clip (up to 30 seconds)	Same foreground-service and indicator rules as the microphone; larger files change the transfer design.
Not planned	Location history, always-on listening, call recording	Excluded by policy, not just deferred. Call recording is additionally restricted by Android and by law in many places.

1.5 Design principles

1. **Request-driven only.** The app does nothing with a sensor until an authenticated request arrives, and stops as soon as that request completes.
2. **Visible by design.** Every capture is announced on the phone by the foreground-service notification and the system privacy indicators, and written to the audit log.
3. **Smallest possible surface.** One app, one listener, one token, three capabilities in v1.
4. **Fail closed.** A missing permission, a disabled toggle or an expired token returns an error. Nothing falls back to a weaker path.
5. **The phone is the record.** Captured media lives on the device until the agent's request completes; the audit log lives only on the device.

2. Architecture

2.1 Shape of the system

Two components only: the Android app on the Pixel, and the agent running on Rahul's existing infrastructure. They meet over the Tailscale network his machines already belong to. There is no server in the middle, no cloud account and no public endpoint.



2.2 Components inside the app

Component	Responsibility
API listener	A lightweight embedded HTTP server (Ktor or NanoHTTPD class) bound *only* to the Tailscale interface address. It must never bind to 0.0.0.0 on Wi-Fi or mobile data interfaces. Default port 8734, configurable at pairing.
Capture service	A foreground service of type `camera | microphone` that owns CameraX for still capture and MediaRecorder/AudioRecord for clips. Started per request, stopped when the response is sent.
Gallery provider	MediaStore queries for image listing and retrieval. Read-only.
Policy store	Per-capability toggles, paired-token hash, and settings, held in app-private storage (EncryptedSharedPreferences / DataStore).
Audit log	Append-only record in app-private storage of every request: time, capability, parameters, result, bytes returned. Viewable and exportable in the app; never transmitted except when the agent explicitly calls the audit endpoint.
Settings screen	A single Activity: pairing status, token rotation, capability toggles, audit-log viewer, listener start/stop.

2.3 Transport: Tailscale only

- The listener binds to the phone's Tailscale address (100.x.y.z) and refuses connections arriving on any other interface.
- Tailscale already encrypts the tunnel with WireGuard and authenticates the peer device. The app's own token (Section 5) is a second, independent layer, so a device merely joining the tailnet does not gain capture rights.
- The phone is addressed by its MagicDNS name, so address changes on the tailnet do not break pairing.
- If Tailscale is stopped or the phone leaves the tailnet, the API is simply unreachable. That is the intended failure mode; the app must not fall back to LAN or public exposure.
- Tailscale ACLs: recommended but optional hardening - restrict the tailnet so only Rahul's agent host can reach the phone on the API port.

2.4 Request lifecycle (camera example)

1. Agent sends `POST /v1/camera/snapshot` with the bearer token.
2. App verifies the token, checks the camera toggle and the CAMERA permission.
3. App starts the foreground capture service; Android posts the persistent notification.
4. CameraX captures one frame; the green camera indicator shows while the sensor is live.
5. App writes the audit entry, stops the service, and returns the JPEG (base64 in JSON for v1; see Section 4.4 for the size trade-off).
6. If any check fails, the app returns a structured error and captures nothing.

2.5 Availability expectations

The listener is a normal Android app component, so Android may stop it when the phone is under memory pressure or when battery optimisation applies. Version 1 accepts this: a failed health check means Rahul reopens the app or the agent reports the phone as unreachable. A "keep the listener running" mode (a persistent low-priority foreground service) is a v1 option behind its own toggle, off by default, because it costs battery and adds a permanent notification. Doze mode may delay requests while the phone is idle and unplugged; the API contract in Section 4 treats slow responses as normal and sets a 30-second client timeout.

3. Android platform constraints

These are properties of the Android platform (the Pixel 10 Pro XL ships with Android 14-class rules and later), not choices this app can negotiate. The specification treats each one as fixed.

3.1 Foreground service requirement

- Camera and microphone access from the background requires a **foreground service**: a running service with a persistent, user-visible notification for the whole capture.
- From Android 14, the service must declare the correct **foreground service types** - `android:foregroundServiceType="camera|microphone"` - in the manifest, hold the matching `FOREGROUND_SERVICE_CAMERA` / `FOREGROUND_SERVICE_MICROPHONE` permissions, and already hold the runtime CAMERA / RECORD_AUDIO permission before the service starts. Starting the service without them throws a system exception.
- A foreground service cannot be started from some background states (for example, certain battery-saver and app-standby conditions). The API returns `service_start_blocked` in that case rather than retrying silently.

3.2 Privacy indicators and permission one-taps

- While the camera or microphone is live, Android shows its **green privacy indicators** in the status bar, and the Privacy Dashboard records the access against this app. There is no supported way to suppress either, and v1 does not try.
- Permissions are granted through the standard runtime permission dialogs, tapped by Rahul on the phone. The agent cannot grant them remotely.
- Android 14's **partial photo access** ("Select photos") applies to the gallery: if Rahul grants selected-photos access, the gallery endpoints return only the selected set. Full access is requested at pairing; the app must behave correctly under either grant and report which mode it is in via the capabilities endpoint.
- If a permission is permanently denied ("Don't ask again"), the app surfaces that state and stops asking; re-enabling is a manual step in system Settings.

3.3 Other platform limits that shape the design

Constraint	Effect on this app
Battery optimisation / Doze	Requests may be delayed while the phone idles. Pairing includes an optional battery-optimisation exemption request, which Rahul approves or declines on the phone.
Screen-off capture	Camera capture with the screen off is unreliable across devices and is not promised in v1. If the device is locked, the snapshot endpoint may return `device_locked`; the mic and gallery endpoints still work while locked.
Auto-revoked permissions	If the app goes unused for months, Android can auto-reset its permissions. The capabilities endpoint reports current permission states so the agent can tell Rahul exactly what to re-grant.
One sensor consumer at a time	If another app holds the camera (for example, a call is using the microphone), capture fails with `sensor_busy`. No pre-emption is attempted.
Sideloading	Installing outside the Play Store triggers the "allow from this source" prompt and a Play Protect scan. Updates are manual and must keep the same signing key, or Android refuses the update.
No silent self-update	The app cannot update itself. Every version change is a deliberate install by Rahul.

> **Plain statement of the trade-off:** this app can take a photo or record a clip when the agent asks, but never invisibly. A notification appears, the green indicator lights while a sensor is live, and the access is logged on the phone and in Android's Privacy Dashboard. That visibility is a feature of the design, and it is also the reason the app can exist at all on a current Pixel.

3.4 Permissions requested - the complete v1 list

Permission	Used by
`CAMERA`	Camera snapshot
`RECORD_AUDIO`	Microphone clip
`READ_MEDIA_IMAGES` (or partial-access equivalent)	Gallery list and fetch
`FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_CAMERA`, `FOREGROUND_SERVICE_MICROPHONE`	Capture service
`POST_NOTIFICATIONS`	Foreground-service and status notifications
`INTERNET`	Binding the local API listener (Tailscale interface only)

4. API design

4.1 Conventions

- Base URL: `http://<pixel-magicdns-name>:8734/v1` over the Tailscale tunnel (encryption is provided by WireGuard; the token below is still mandatory).
- Every request carries `Authorization: Bearer <token>`. Requests without a valid token get `401` and are written to the audit log as rejected attempts.
- Requests and responses are JSON, except that image and audio payloads travel base64-encoded inside the JSON body in v1 (Section 4.4).
- Every response carries a `request_id` that also appears in the on-device audit log, so any result can be traced to a log entry.
- Versioning is by URL prefix (`/v1`). Breaking changes ship as `/v2` alongside, never by silently changing v1 shapes.

4.2 Endpoints

Endpoint	Purpose
`GET /v1/health`	Liveness, app version, listener uptime, battery level and charging state. The only endpoint the agent polls.
`GET /v1/capabilities`	Per-capability state: toggle on/off, permission granted/denied, gallery access mode (full or selected), foreground-service readiness.
`POST /v1/camera/snapshot`	Capture one still photo and return it.
`POST /v1/mic/record`	Record one clip of a requested duration and return it.
`GET /v1/gallery/recent?limit=20`	List recent photos (metadata only, no image bytes).
`GET /v1/gallery/photo/{id}`	Fetch one photo by its MediaStore ID, optionally downscaled.
`GET /v1/audit?limit=50`	Read the most recent audit-log entries.

4.3 Request and response shapes

Camera snapshot

```
POST /v1/camera/snapshot
{ "camera": "rear", "flash": false, "max_width_px": 2048 }

200 OK
{ "request_id": "a1b2c3",
  "captured_at": "2026-10-26T20:41:03+02:00",
  "width_px": 2048, "height_px": 1536,
  "format": "jpeg", "size_bytes": 812340,
  "data_base64": "..." }
```

Microphone clip

```
POST /v1/mic/record
{ "duration_seconds": 30 }

200 OK
{ "request_id": "d4e5f6",
  "recorded_at": "2026-10-26T20:42:10+02:00",
  "duration_seconds": 30, "format": "m4a",
  "sample_rate_hz": 44100, "size_bytes": 241208,
  "data_base64": "..." }
```

Duration rules: minimum 1 second, maximum 120 seconds; out-of-range values are rejected with `invalid_parameter`, never silently clamped, so the agent always knows exactly what was recorded.

Gallery list and fetch

```
GET /v1/gallery/recent?limit=3

200 OK
{ "access_mode": "full",
  "photos": [
    { "id": "10024", "taken_at": "2026-10-26T19:05:11+02:00",
      "width_px": 4080, "height_px": 3060,
      "size_bytes": 3102455, "mime": "image/jpeg" } ] }

GET /v1/gallery/photo/10024?max_width_px=1600
200 OK -> same envelope as the snapshot response,
         with "source": "gallery" and the original taken_at.
```

4.4 Media transfer - the v1 trade-off

Base64 inside JSON inflates payloads by roughly a third, and a full-resolution Pixel photo can exceed 4 MB. Version 1 accepts this for simplicity, with two guardrails: snapshot and gallery fetch accept `max_width_px` so the agent normally requests a downscaled copy (the original stays untouched in the gallery), and responses above 12 MB are refused with `payload_too_large` plus the photo's ID, so the agent can re-request it smaller. A binary-body transfer mode is the first scheduled API improvement if v1 proves slow in practice.

4.5 Error vocabulary

Errors are JSON with a stable `error` code, a human-readable `message`, and the `request_id`. The agent must handle every code below; there is no generic retry-everything path.

HTTP	Code	Meaning and expected agent behaviour
401	`unauthorized`	Token missing, wrong or revoked. Stop; tell Rahul pairing needs attention.
403	`capability_disabled`	The toggle for this capability is off on the phone. Report it; do not retry.
403	`permission_denied`	The Android runtime permission is not granted. Name the exact permission Rahul must grant.
409	`sensor_busy`	Another app holds the camera/microphone, or a capture is already running. One retry after a short wait is reasonable.
409	`device_locked`	Camera requested while the device is locked. Ask Rahul to unlock, then retry.
422	`invalid_parameter`	Bad duration, unknown camera, or similar. Fix the request; never retry unchanged.
413	`payload_too_large`	Result exceeds the transfer cap. Re-request with a smaller `max_width_px`.
404	`photo_not_found`	Gallery ID no longer exists (deleted or outside selected-photos access).
503	`service_start_blocked`	Android refused to start the foreground service. Surface it; Rahul may need to open the app once.
504	`capture_timeout`	Sensor did not produce a result in time. The partial file, if any, is deleted on the phone.

5. Security and consent model

5.1 Who can ask, and how that is enforced

- **Network layer:** the listener answers only on the Tailscale interface, so a request must already come from a device on Rahul's tailnet.
- **Token layer:** every request needs the bearer token. The token is a 256-bit random value generated on the phone during pairing, shown once as text and a QR code, and stored by the app only as a salted hash. The agent keeps its copy in the same protected credential storage it uses for other service secrets.
- **Capability layer:** even a valid token cannot use a capability whose toggle is off. Toggles are set only on the phone.

5.2 Per-capability toggles

Camera, microphone and gallery each have an independent switch on the settings screen, plus a master switch that stops the listener entirely. Defaults at first install: all three capabilities **off**, listener **off**. Pairing turns the listener on; Rahul turns each capability on deliberately. Turning a toggle off takes effect on the next request - an in-flight capture finishes, since interrupting a recording mid-write risks corrupting the file, and the change is audit-logged.

5.3 The audit log is the consent record

- Every request - successful, failed or rejected - appends an entry: timestamp, endpoint, parameters (never media content), result code, bytes returned, and the request ID.
- The log is append-only from inside the app; entries can be exported or cleared by Rahul from the settings screen, and clearing the log is itself recorded as the first entry of the new log.
- Retention default: the last 1,000 entries. Rahul can review the log on the phone at any time without the agent's involvement.
- Android's own Privacy Dashboard independently corroborates camera and microphone accesses, so the app's log is not the only record.

5.4 Token provisioning, rotation and revocation

1. **Provision:** during pairing, the app generates the token and displays it once. Rahul transfers it to the agent through the pairing session; it is never sent by email or chat in plain text afterwards.
2. **Rotate:** one tap on the settings screen generates a new token and invalidates the old one immediately. Rotation is the response to any suspicion the token leaked.
3. **Revoke:** a separate "unpair" action deletes the token hash and stops the listener. After unpairing, no request can succeed until a fresh pairing is done in person on the phone.
4. **Rate limiting:** the listener allows at most 10 capture requests per minute and 5 failed-auth attempts per minute per source; beyond that it returns `429`. This bounds both accidents and probing.

5.5 If the phone is lost

1. Removing the phone from the Tailscale network (from another signed-in device or the Tailscale admin console) cuts the only network path to the app. This is the first move and needs nothing on the phone itself.
2. Android's Find My Device can lock or erase the phone; erasing removes the app, its token hash and its audit log.
3. The agent-side token copy is deleted or rotated out at the same time, so even a recovered phone paired to a re-added tailnet device would still fail authentication.
4. Because media is returned per request and not stored in the app beyond the gallery itself, a lost phone exposes no separate archive of agent-requested captures inside the app.

5.6 Data handling

- Photos taken through the snapshot endpoint are **not** added to the phone's gallery by default, so agent-requested captures do not mix into Rahul's personal photos. A pairing-time setting can change this if he prefers them saved (Open Question 3).
- Temporary capture files are deleted from app storage immediately after a successful response, and also after any failure.
- The app contains no advertising, analytics or third-party SDKs, and makes no network calls other than answering API requests.

6. Build and install plan

6.1 Build environment (on the agent's VM)

- Toolchain: JDK 17, Android SDK command-line tools with platform API 35 and build-tools, Gradle via the project wrapper. The build is a standard `./gradlew assembleRelease` producing a single APK; no build step needs the phone or any app-store account.
- Language and libraries: Kotlin, CameraX for still capture, MediaRecorder for audio, Ktor server (or equivalent lightweight embedded server) for the listener, DataStore for settings. All are mainstream, maintained libraries; no experimental dependencies in v1.
- The project lives in Rahul's workspace as source he owns, so any later change is a rebuild and reinstall, not a new project.

6.2 Signing

- A dedicated release keystore is generated once, on the VM, with a strong password held in protected credential storage - never in the project folder or in chat.
- The same key signs every build. This matters: Android will only install an update over an existing install when the signature matches, and a lost key means uninstalling (losing settings, pairing and the audit log) before a differently signed build can be installed.
- A backup of the keystore is part of the build deliverables, stored with Rahul's other recovery material.

6.3 Install on the Pixel

1. Transfer the APK to the phone (USB, or a download from a private link Rahul controls).
2. Open it and work through the install prompts: allow installs from that source for this one install, then let Play Protect complete its scan.
3. Open the app; confirm the listener is off and all capability toggles are off, as shipped.

6.4 Pairing and onboarding - one supervised session

Pairing is done once, with the phone in Rahul's hand. Expected time: about ten minutes.

1. **Confirm Tailscale:** verify the phone is signed in to the same tailnet as the agent host and note its MagicDNS name.
2. **Grant permissions:** start the listener (it starts in pairing mode, accepting only the pairing call, for 5 minutes). The app walks through the permission dialogs in order - notifications, camera, microphone, photos (full or selected, his choice). Each grant is Rahul's tap.
3. **Generate the token:** the app generates the token and shows it as text and a QR code; the agent stores its copy in protected credential storage, and the health endpoint is called to prove the path end to end.
4. **Set the toggles:** Rahul enables exactly the capabilities he wants live, and decides the battery-optimisation exemption prompt.
5. **Smoke test:** one snapshot, one 5-second clip and one gallery fetch are run and verified together, and the three audit entries are reviewed on the phone. This test doubles as acceptance for Milestone 5.

6.5 Updates and uninstall

- Updates: build the new APK with the same key, transfer, install over the top. Settings, pairing and the audit log survive. A release note in the app states what changed.
- Uninstall: removing the app removes the listener, token hash, settings and audit log in one step. Nothing outside the app needs cleanup except deleting the agent's stored token copy.
- Rollback: keeping the previous signed APK on the VM allows reinstalling the earlier version's code; Android may refuse a downgrade over a newer install, in which case rollback means uninstall and re-pair.

7. Milestones for v1

Milestone	Deliverable	Acceptance test (pass / fail)
M1 - Skeleton	App installs on the Pixel; listener binds to the Tailscale interface only; health endpoint answers	Health call succeeds over the tailnet and fails from a non-tailnet network and from the phone's Wi-Fi address.
M2 - Pairing	Token generation, hashed storage, auth middleware, rate limiting, settings screen with toggles	Requests with no token and with a wrong token both return 401 and appear in the audit log as rejected.
M3 - Camera	Foreground capture service with declared types; snapshot endpoint	A snapshot returns a viewable JPEG while the notification and green indicator are visible; with the camera toggle off, the same request returns capability_disabled and captures nothing.
M4 - Microphone and gallery	Record endpoint with the 120-second cap; gallery list and fetch with full and selected access modes	A 30-second clip plays back intelligibly; a 121-second request is rejected; gallery fetch returns the requested photo and reports the correct access mode.
M5 - Hardening and handover	Audit-log viewer and export, unpair action, battery behaviour check, signed release build and keystore backup	The supervised pairing session in Section 6.4 completes, including the three-request smoke test and audit review.

Milestones are sequential; M3 is the risk milestone, because it is where foreground-service behaviour on the real device is proven. If M3 exposes a device-specific problem, the schedule slips there rather than in hardening.

7.1 What "done" means for v1

- All five acceptance tests pass on Rahul's actual Pixel, not an emulator.
- The audit log on the phone shows every test access, and nothing else.
- The signing keystore backup exists and its location is recorded.
- Rahul has seen the notification, the green indicator and the Privacy Dashboard entry for a real capture, and is satisfied with how visible capture is.

8. Open questions for Rahul

Two questions block the build (marked with an asterisk); the rest have defaults in this specification that stand unless he overrides them.

1. *** Listener availability:** should the listener run only while the app is open or recently used (v1 default, best battery), or should the always-on foreground-service mode be built in v1 so the agent can reach the phone at any time, at the cost of a permanent notification and some battery?
2. *** Gallery access level:** full photo access, or Android's selected-photos mode with a curated set? Full access is what makes "fetch my last photo" work for any photo; selected mode is the smaller grant.
3. **Snapshot destination:** the specification defaults to *not* saving agent-taken photos into the gallery. Should they be saved there instead, so there is a personal copy of everything the camera captured?
4. **Microphone cap:** 120 seconds is the v1 ceiling. Is a longer cap (for example, 5 minutes for dictated notes) wanted, knowing the clip returns as one file?
5. **Default camera and quality:** rear camera at up to 2048 px wide is the default. Is full-resolution return needed often enough to justify the larger transfers?
6. **Audit retention:** the last 1,000 entries is the default. Should the log also be exportable automatically to the agent on a schedule, or stay strictly on the phone until requested?
7. **Phase 2 appetite:** which of the deferred capabilities matter enough to schedule next - sensors snapshot, screen capture (with its per-session consent prompt), or short video?

> **Next step:** answers to Questions 1 and 2, plus a go-ahead, are enough to start Milestone 1. Everything else can be decided while the skeleton is being built.

Appendix A - Manifest sketch (for the builder)

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_CAMERA" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_MICROPHONE" />
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
<uses-permission android:name="android.permission.INTERNET" />

<service android:name=".capture.CaptureService"
    android:foregroundServiceType="camera|microphone"
    android:exported="false" />
```

This sketch is orientation for whoever builds the app, not code. The API listener is an internal component and is never declared as an exported Android component; network reachability exists only through the Tailscale interface binding described in Section 2.